Security Now! #1083 - 06-16-26

Patch Tuesday à la AI

This week on Security Now!

• Rootkits found in more than 400 ArchLinux User Repository packages. • The US government requests Anthropic to remove Mythos and Fable. • CISA responds to AI-driven attacks with new patching requirements. • NPM to switch to more secure install defaults. Will it help?
• Our listeners react to last week's PHP commentary. • June shows that AI has arrived for vulnerability discovery.

A loop with the branch test at the end

Security News

ArchLinux User Repository (AUR) compromised

The Arch Linux Repository (AUR) was found to have been compromised with more than 400 instances of Linux rootkit and infostealer malware. The infostealer targets credentials and access tokens. Last Thursday, the site "ioctl.fail" posted their analysis of the infiltration campaign. They open their report by explaining:

> *This report summarizes static reverse engineering of the Linux ELF malware sample named "deps" and static review of the recovered npm package source associated with the incident. The sample and package were treated as malicious throughout handling.No dynamic execution of the ELF, npm package, lifecycle scripts, or package code was performed. The binary is stripped and implemented with Rust-style async state machines. Function names in this report are analyst-assigned names based on decompiled behavior.*

So, what they're saying here is that at no point did they execute the malware under any context. The "ELF" referred to here and throughout for those not versed in Linux lingo is the abbreviation of Executable and Linkable Format — thus ELF. It's a very flexible format that's used almost universally by Linux and the UNIXes and embedded RTOSes and pretty much everywhere other than Windows, which uses the PE — Portable Executable — format and Mac which uses Mach-O.

In any event, they chose not to let this thing loose in any sandbox, such as a secure virtual machine, where any damage it might attempt to do could be observed and contained. Instead, they reverse-engineered the malware from a static sample of the malware's code. And since the malicious binary had been stripped of any latent symbolic names, such as variables or function call names, the analyst assigned the names themselves once a function's purpose became clear. They then explain:

> *This sample was recovered from a supply-chain compromise involving an Arch User Repository (AUR) package build flow. In the reported intrusion path, the attacker modified AUR build steps so that the build process downloaded and installed a malicious npm package. That package masqueraded as atomic-lockfile version 1.4.2 and included the Linux ELF payload at src/hooks/deps.*
>
> *The malicious npm package used a preinstall lifecycle hook to execute the ELF automatically during npm installation. This means a developer workstation, a maintainer's machine, or CI/build host could execute the malware as a side effect of building or installing the compromised AUR package. "deps" is a Linux credential stealer with optional root-only eBPF rootkit capabilities.*

eBPF refers to the "extended Berkeley Packet Filter" technology that allows user-provided code to be run in the Linux Kernel. This gives that code direct access to the most privileged information in the system. They continue to describe the malware, writing:

> *It is designed for developer workstations and build environments. It targets browser and Electron application data, Slack, Microsoft Teams, Discord, GitHub, npm, Vault, Docker/Podman, SSH, VPN material, shell histories, and other local developer secrets.*

Yikes! In other words, no developer wants this anywhere near them.

The recovered supply-chain package identifies itself as atomic-lockfile version 1.4.2. It contains a malicious npm lifecycle entry: "preinstall": "./src/hooks/deps". That lifecycle script executes the ELF directly during npm installation when lifecycle scripts are enabled. The ELF in the package source is byte-identical to the analyzed sample.

The attacker-controlled C2 (Command & Control) endpoint was recovered from the ELF. It is not supplied by the npm package, command-line arguments, or a JavaScript wrapper. The binary decodes an onion service address at runtime:

olrh4mibs62l6kkuvvjyc5lrercqg5tz543r4lsw3o6mh5qb7g7sneid.onion

The command/result callback is POST /api/agent, sent through a local loopback/SOCKS-style transport. The local 127.0.0.1 traffic is an intermediate transport layer, not the attacker endpoint. When eBPF is available, the malware can hide local process and socket artifacts used by that transport.

In other words, if the code has access to eBPF, that access will be used in a rootkit-like manner to completely obscure any and all evidence of any infection. Without eBPF, the local network presence of "something" could be observed.

The recovered package source appears to be a mostly legitimate TypeScript npm package with a malicious ELF inserted into the source tree and wired into npm lifecycle execution. Static review of the package source outside the ELF found no JavaScript wrapper, no additional C2 configuration, no command-line arguments passed to the ELF, and no package-layer references to temp.sh, /api/agent, Discord webhooks, or a public C2 domain/IP.

Conclusion: the malicious npm package provides the execution vector. The C2 endpoint is encoded inside the ELF itself.

So now we know what it is and how it's delivered and carried into the system... which brings us to: "What does it do inside the developer's machine once it takes hold?" where we learn why no developer wants to have this anywhere near their system. It:

- *Installs persistence using root or per-user systemd service units.*
- *Enforces a single active instance using flock().*
- *Redirects standard input/output/error to /dev/null.*
- *Ignores SIGPIPE.*
- *Reads /proc/self/exe to locate and copy/install its current executable.*
- *Uses Rust async runtime logic to run collectors and transport tasks.*
- *Enumerates Chromium-family browser profiles and Electron app data.*
- *Reads SQLite cookie databases and LevelDB local storage.*
- *Extracts Chromium/Electron cookies and service tokens.*
- *Queries Slack, Microsoft Teams, Discord, GitHub, npm, and OpenAI/ChatGPT APIs with stolen tokens or cookies.*
- *Searches local filesystem locations for SSH keys, shell history, Vault tokens,*

Docker/Podman credentials, VPN material, and developer secrets.

- *Uploads file content to temp.sh.*
- *Calls back to the recovered onion C2 over POST /api/agent.*
- *Uses a local loopback/SOCKS-style transport before reaching proxied destinations.*
- *Includes a downloader/stager path tied to /usr/bin/monero-wallet-gui.*
- *If sufficiently privileged, loads an embedded eBPF rootkit to hide processes, process names, and socket inodes.*

And, just to reiterate, more than 400 instances of Arch Linux Repository packages have been found infected with this nastiness.

We've referred tangentially to infostealer malware from time to time. It's an increasingly prevalent form of malware because its goal is obtaining information that would allow an attacker to pivot to some other target. The malware might get into a developer's machine who happens to have AWS credentials for some other juicy target. So the bad guys are less interested in the initial victim than in what other systems or networks that victim might have access to.

Since we've never looked closely at infostealers, since they are definitely something that no one wants to discover in their systems, since they are growing in prevalence, and since we have a very nicely reverse-engineered infostealer here, and I want to share the details of what info this representative infostealer steals:

The malware targets developer and collaboration data:

Browsers and Chromium Profile Stores: Google Chrome, Chrome Beta, Chrome Dev, Microsoft Edge, Edge Beta, Edge Dev, Brave, Brave Beta, Brave Nightly, Vivaldi, Opera, Opera Beta, Opera Developer, Yandex Browser, Epic Privacy Browser, Iridium, Ungoogled Chromium, Thorium, Comodo Dragon, SRWare Iron, Cent Browser, Slimjet, Maxthon, UC Browser, CocCoc, Naver Whale, Chromium Flatpak, Google Chrome Flatpak, Microsoft Edge Flatpak, Brave Flatpak, Vivaldi Flatpak, Opera Flatpak, and Yandex Browser Flatpak

Profile artifacts targeted include: Local Storage/leveldb, Network/Cookies, Cookies, Default/Cookies and Chromium encrypted cookie values

Collaboration and Electron Applications: Slack, Slack Flatpak, Slack Snap, Microsoft Teams, Microsoft Teams legacy stores, Microsoft Teams Flatpak/browser-derived stores, Discord, Discord PTB, Discord Canary, Discord Flatpak variants, Discord Snap variants, Vesktop, Legcord, WebCord, ArmCord, Vencord, NativeCord, Abaddon, Dissent, Ripcord, and Datcord

*Confirmed Slack paths and data include: .config/Slack, .var/app/com.slack.Slack/config/Slack, snap/slack/current/.config/Slack, Slack d cookies for *.slack.com, Slack API enrichment through /api/auth.test, /api/users.info, and /api/conversations.list*

Confirmed Microsoft Teams and Microsoft service artifacts include: .config/Microsoft/Microsoft Teams, authsvc.teams.microsoft.com, teams.microsoft.com, skypeToken, regionGtm, cache.token, Authorization: Bearer, X-Skypetoken, Teams account, tenant, and team metadata.

Confirmed Discord artifacts include: Discord tokens from Electron/browser storage,

/api/v9/users/@me, /api/v9/users/@me/guilds?with_counts=true, user ID, phone, MFA state, premium/Nitro type, flags, guild ownership, permissions, and member-count metadata.

Developer Accounts and Package Ecosystems: GitHub, npm, and OpenAI/ChatGPT account metadata.

Confirmed GitHub strings and endpoints include: api.github.com, GET /user HTTP/1.1, GET, /user/repos?sort=stars&direction=desc&per_page=3&type=owner, Authorization: Bearer, User-Agent: git/2.39.0, Bad credentials, account and repository metadata such as login, company, public repository count, followers, and repository stars.

Confirmed npm strings and endpoints include: registry.npmjs.org, GET /-/whoami, GET /-/v1/search?text=maintainer%3A...&size=3, package publishing identity and maintainer package metadata.

The OpenAI/ChatGPT path queries api.openai.com with stolen bearer material for account metadata. This is credential validation/enrichment against a third-party service, not evidence that OpenAI is attacker-controlled infrastructure.

Local Developer Secrets: Vault token files, Docker command history and registry credential material, Podman command history and registry credential material, SSH keys and SSH configuration, PuTTY private-key material, VPN profiles and .ovpn files, shell histories for bash, zsh, and fish, command history containing sftp, ssh-keygen, ssh-copy-id, ssh-add, rsync, putty, plink, docker, docker-compose, and podman commands.

I know that was a lot, but it's important to appreciate that the more than 400 instances of this malware which were discovered residing in the Arch Linux Repository, were expressly designed to root out any and all instances of any of that developer data and send it back to its command and control server. This stuff is out there and developers really need to be more cautious than ever.

US government "requests" Anthropic take Claude Fable 5 and Mythos 5 down

Last Friday afternoon, at the request of the U.S. government's nonspecified concerns for national security, Anthropic shut down all access to their two most advanced models: Claude Fable 5 and Mythos 5. We should note that claiming "national security" has become the catch-all phrase used by the U.S government which should be taken to either mean "because it's what we want" or "because we say so." It's often not very satisfying. But in any event, since it was at the start of last week's podcast that Claude Fable 5 first appeared and Leo shared with all of us that it appeared to be another major leap forward, I want to share what Anthropic posted about why their two newest models had been taken down.

As I'm sharing this, listen to the language Anthropic uses when they talk about safeguards and jailbreaks. It echoes the position everyone here has heard me articulate many times before, which is that our current LLM technology is almost certainly going to be hostile to any form of control — the way it operates makes it inherently uncontrollable. The headline of Friday's posting was: *"Statement on the US government directive to suspend access to Fable 5 and Mythos 5."* Anthropic wrote:

The US government, citing national security authorities, has issued an export control directive to suspend all access to Fable 5 and Mythos 5 by any foreign national, whether inside or outside the United States, including foreign national Anthropic employees. The net effect of this order is that we must abruptly disable Fable 5 and Mythos 5 for all our customers to ensure compliance. Access to all other Anthropic models will not be affected.

We received the directive from the government today at 5:21pm (ET). The letter did not provide specific details of its national security concern. Our understanding is that the government believes it has become aware of a method of bypassing, or "jailbreaking" Fable 5. We reviewed a demonstration of this specific technique being used to identify a small number of previously known, minor vulnerabilities. These vulnerabilities all appear relatively simple, and we have found that other publicly-available models are able to discover them as well without requiring a bypass.

Anthropic's posture with respect to Fable's safeguards, as laid out in our launch blog post, is the following:

- *We have instituted strong safeguards that greatly reduce the likelihood that Fable is misused for tasks related to cybersecurity (among others). In fact, our safeguards are so strong that many users have complained that they are overly broad.*

- *In the weeks leading up to the launch of Fable, Anthropic worked with the US government, the UK AISI, multiple private third-party organizations and internal teams to red-team Fable's safeguards for thousands of hours in total.*

- *These tests showed that Fable's safeguards are substantially more effective than those of any previously deployed model.*

- *No testers have yet been able to find a universal jailbreak—a jailbreak method that can very broadly bypass the model's safeguards, unblocking a wide range of cyber capabilities.*

- *We suspect that perfect jailbreak resistance is not currently possible for any model provider. Every safeguard used in the industry is vulnerable to non-universal jailbreaks (which can elicit some cyber information in specific circumstances), and it is likely that universal jailbreaks will eventually be found in the future. We stated this clearly when we released Fable 5.*

- *Given that perfect jailbreak resistance does not appear to be possible today, Anthropic adopted a defense in depth strategy with Fable 5. We aimed to make jailbreaks either narrow (in the case of non-universal jailbreaks) or very expensive to produce (in the case of universal jailbreaks), and to combine this with thorough monitoring to quickly detect and shut down any successful attacks. This is also why Anthropic has required 30-day retention of customer data with Fable—a policy change that carries real costs for us with customers, but that allows us to research and mitigate jailbreaks.*

- *We stand by this defense in depth strategy. It reduces the risks posed by Fable, making them comparable to the risks of existing models already deployed across the industry.*

- *We have not even received a disclosure of a concerning non-universal potential jailbreak that led to a harmful result. The potential jailbreaks that have been disclosed to us are either entirely benign responses or are minor findings that provide no Mythos-specific uplift.*

To date, the government has only given us verbal evidence of a potential narrow, non-universal jailbreak, which essentially consists of asking the model to read a specific codebase and fix any software flaws. Our understanding is that one potential jailbreak was shared with the government. We have reviewed a report that we believe is the basis of the government's directive and validated that the level of capability displayed there is widely available from other models (including OpenAI's GPT-5.5), and is used every day by the defenders who keep systems safe. We will share more details over the next 24 hours.

We are complying with the government's legal directive and are removing access to Fable 5 and Mythos 5 for all users. However, we disagree that the finding of a narrow potential jailbreak should be cause for recalling a commercial model deployed to hundreds of millions of people. If this standard was applied across the industry, we believe it would essentially halt all new model deployments for all frontier model providers.

As we have stated publicly, we believe the government should have the ability to block unsafe deployments, as part of a statutory process that is transparent, fair, clear, and grounded in technical facts. This action does not adhere to those principles.

We apologize for this disruption to our customers. We believe this is a misunderstanding and are working to restore access as soon as possible.

I suppose this is the flip side consequence of all the marketing attention – which many have labeled "hype" – that Anthropic deliberately created to accompany the restricted-access to Claude Mythos Preview. They warned that it was too powerful to widely release. Then, along comes Fable 5, which they said was even stronger. But when they then release both Mythos 5 and Fable 5, there are those in the government who do not want this domestically developed AI – if we take the government at face value – to be used by any foreign national. Since there's no practical way for Anthropic – or anyone else for that matter – to control who has access to what on the Internet, both models must be entirely removed from Internet access.

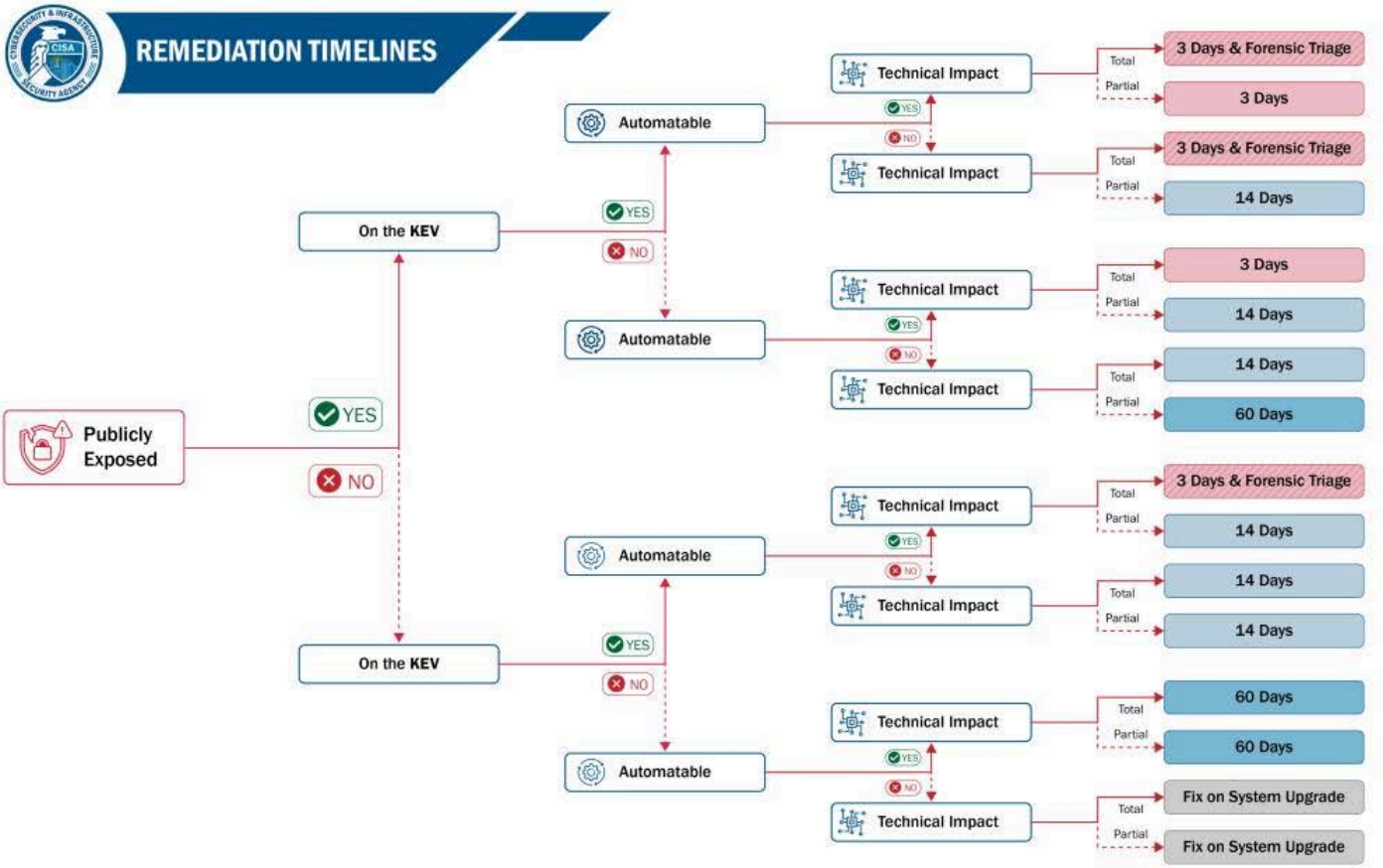
How this will end I have no idea... but it sure is interesting to watch.

CISA responds to the expected speed of AI-driven attacks

As we saw last week, the change to the historical attack pattern that was already accelerating, prior to the rise of AI, will be access to significantly more advanced autonomous AI-attacking capabilities by a much wider range of attacker. And this will, for the first time, include those who are far less knowledgeable and skilled. Just as we've seen many of this podcast's non-coders delighting in their new found AI-driven ability to code, it's only a matter of time – and probably not much time – until we see attacks managed by non-hackers using next-generation AI-driven autonomous attack frameworks. Last week's report from Anthropic's red showed one instance of this that was, by the end of March, still the exception. But a year from now it will be commonplace.

Against this backdrop, last Wednesday on June 10th, CISA released *"Binding Operational Directive 26-04: Prioritizing Security Updates Based on Risk."* This BOD formalizes the timeline under which federally managed IT systems are now required to respond to cyber threats that

CISA identifies. We've already seen and commented about the response speed required by CISA to some recent threats. But now that somewhat breathtaking 3-day-to-patch timeline is being formalized:



The guidance has been reduced to a tree with four binary decision points. Since 2 to the power of 4 is 16, there are 16 leaves at the end of this decision tree. The four branching decisions are: Has the vulnerability been publicly released? Is the vulnerability in KEV, the Known Exploited Vulnerabilities list? Is exploitation of the vulnerability automatable by the adversary? And is the impact of its exploitation partial or total control? I've reproduced the new CISA remediation timeline tree in the show notes. Each one of the sixteen branches ends in 3 days to patch, 14 days to patch, 60 days to patch, or patch on system upgrade. So, it's essentially, patch it immediately; you have two weeks to patch; you have two months to patch; or don't worry about it much.

So, for example, riding along the worst-case branches of the tree, a known vulnerability that's been publicly exposed, is on the KEV list, can be automated and carries an impact of "total control" by the attacker must be fixed within 3 days. And following the best-case branches, if the vulnerability has not been publicly exposed, is not on the KEV list, cannot be automated and regardless of whether or not its impact would be partial or total control or the victim's system, there's no hurry and it can be fixed during routine system updates.

What this means is that all federal agencies falling within CISA's Binding Operational Directives will need to put a system in place to push updates out to their equipment with far higher priority than ever before. Maintaining up-to-date systems is no longer something that government

agencies will be able to give lip service to while planning to "get around to updating eventually." Those lazy days are over... and I'd be pretty certain that they're never coming back since once those systems have been painfully put in place, why would they not continue?

A change coming to NPM install defaults

The news is that the much-attacked npm (Node.js Package Manager) will flip its defaults to begin disabling the auto-running of installation scripts starting with the July 2026 release of its version 12.0. The change hopes to counter the massive rise in the number of supply-chain attacks taking place on the platform. As we've seen here, threat actors are increasingly hiding malicious commands inside install scripts that get auto-executed when a victim installs a new package. This sounds welcome and great. But what effect will it have? I tracked down an informed opinion of someone who should know, writing for the blog at "opensourcemalware.com" to get a reality-check. The posting was titled: *"NPM disables install scripts by default, but is that going to solve its malware problem?"* and the blog's tagline was: *"NPM announced that the new version of the NPM package manager (v12) will come with several security improvements, including disabling install scripts. Is this a game-changer or security theatre?"* Its author wrote:

On June 9, NPM announced the breaking changes coming in v12, and I'm genuinely excited about this announcement! Three permissive defaults that have shipped malware to developers for a decade are about to flip to deny-by-default. Pair that with install-time cooldowns landing across every package manager and the rise of commercial supply-chain firewalls, and you'd be forgiven for thinking the NPM malware problem is finally getting solved.

A year ago I stood on a DEFCON stage and walked through why NPM install scripts are terrible. So let me be the first to say it: these coming v12 changes are good. They are also, in part, theatre — and understanding which part is which is important.

NPM v12 flips three dangerous defaults. The big one is "install scripts". Today, npm install happily runs preinstall, install, and postinstall hooks from any package in your tree, including transitive dependencies you've never heard of. In v12, that stops by default — no automatic lifecycle scripts, no native node-gyp builds, no prepare scripts from git, file, or link dependencies. You opt packages back in with npm approve-scripts, discover what's affected with npm approve-scripts, the --allow-scripts-pending option, and block the rest with npm deny-scripts.

The second change makes git dependencies require an explicit --allow-git option, closing a code-execution hole where a git dependency's own .npmrc could override the git executable — a bypass that worked even with the --ignore-scripts option set.

The third change makes remote URL (HTTPS tarball) dependencies require an --allow-remote option.

These three changes ship around July 2026 and you can prepare today in NPM 11.16.0+. At the same time other security improvements are taking hold

"Cooldowns" are also now everywhere. NPM shipped with support for "min-release-age" in 11.10.0 back in February. This is a setting that refuses to install a version until it's been public for a configurable number of days. The logic is simple and effective: compromised releases usually get caught and pulled within hours, so a short delay filters most of them out at the

install layer with zero scanning. It's worth noting that NPM was the last to this party. pNPM shipped minimumReleaseAge in 10.16 in September 2025, yarn added npmMinimalAgeGate in 4.10.0 the same month, and bun followed in 1.3. There's even an open proposal to make seven days the default, which is the right instinct — almost nobody needs a package the instant it's published.

Supply-chain firewalls have become a product category. Developers are finally installing something on their machines that address malicious package threats. Datadog released an open-source supply-chain-firewall in 2024, and Liran Tal released his tool npq back in 2017. Tools like these wrap package managers and check to see if the user is about to install vulnerable or malicious packages; and if so, they'll block them from being installed. There has been a lot of new tooling in this domain recently with Socket, Aikido, and Endor Labs all offering products in this space. "Package Firewalls" like these work, but of course they rely on developers to not bypass their controls to install malicious packages.

I don't want to be cynical about the right things. Killing automatic install scripts is the single most important change NPM could make. Install hooks are the mechanism behind a huge share of the incidents we document at OpenSourceMalware — they're how a poisoned transitive dependency gets to run arbitrary code on a developer laptop or a CI runner the instant it's pulled. The security community and pnpm have been arguing for deny-by-default here for years. NPM finally agreeing is a genuinely good day.

Cooldowns are the cheapest high-value control in the entire ecosystem. Firewalls and package managers give teams a real shot at stopping a supply chain attack before it lands. None of this is fake security. The problem is what happens next.

The first observation is that "off-by-default" only works if it stays off. The thing the announcement glosses over is that disabling install scripts is going to break a lot of stuff. An enormous amount of legitimate software gets installed, built, and wired into your application environments via install scripts. This, of course, explains why they're such a strong attack vector. There's a simplistic narrative going around that lifecycle scripts only exist to do bad things, and that's just not true.

NPM packages are not just a way to import libraries. Many people, including me, build CLI tools to do necessary utilitarian functions, and many of those tools use install scripts. Some examples of popular packages that use lifecycle scripts are:

- *esbuild (200 million weekly downloads)*
- *sharp (60 million weekly downloads)*
- *core-js (40 million weekly downloads)*
- *puppeteer (10 million weekly downloads)*
- *Nx (9 million weekly downloads)*
- *bufferutil (6 million weekly downloads)*
- *utf-8-validate (8 million weekly downloads)*
- *bcrypt (5 million weekly downloads)*

Additionally, native modules need node-gyp to compile against your platform at install time. Tools that download a platform-specific binary, generate a config, register a shell completion, or build a native addon all lean on install scripts to do their job. This isn't an edge case — it's a meaningful slice of the most-depended-on packages in the registry. The day v12 lands, those packages stop working until someone approves them.

If anyone listening is thinking to themselves “convenience vs security” you are exactly correct. I’m sure that many of us who have used package managers have been somewhat amazed by the astounding degree of automation. You fire it up and stuff happens, packages are grabbed from here and there, compilations are run and linked together, packages are installed, and everything just whizzes by on the console. So, what happened during all that? Who knows? Someone somewhere presumably knows. But the entire point is that we, the developer or end-user who invoked the package manager don’t need to know. It all just magically works. And that is also, of course, the Achilles heel of the entire process... because that extreme magical ease-of-use can be – and increasingly is being – abused to do malicious things behind our backs. We have no idea what’s happening in the first place... so how would we know if something bad was happening?

The problem with flipping these magical defaults off is that the magic, upon which we have become dependent, breaks. The blog post continues:

> *So what will developers actually do? They'll approve. Then they'll approve again. And by the third time a build breaks at 5pm because some transitive dependency needs its postinstall hook, the approval becomes a reflex. The deny-by-default protection quietly degrades into a click-through prompt, and a click-through prompt that fires constantly trains people to click through. We've watched this movie with browser permission dialogs and OS security prompts. There's no reason to expect NPM's to end differently. The control is real; the human standing in front of it is the same human who has a deadline.*
>
> *If you really want to know if disabling install scripts will have the intended effect, you can look to the VSCode ecosystem. With VSCode before version 1.109, the global allowAutomaticTasks setting was enabled by default. This meant that malicious task files would automatically run if victims opened source code that included those task files. Microsoft changed and disabled this feature to be disabled by default in January 2026 after months of threat actors used malicious tasks files to compromise developers.*
>
> *Did that stop threat actors from continuing to use VSCode tasks? Nope. North Korean threat actors continue to use malicious VSCode tasks files as many developers have re-enabled the feature, or other developer tooling has enabled it. In fact, most of the large scale attacks we've seen in 2026 leverage VSCode tasks and settings files to help re-distribute the attack artifacts.*
>
> *The second observation is that the bad guys will find a way: Just because NPM won't run scripts at install time doesn't stop users from running those scripts the second those packages are installed. Even worse, if you already use a library and it's compromised, you don't need to run install scripts to receive the payload. It's going to run in your application. No scripts needed.*
>
> *Now follow the incentives one step further. If install scripts are switched off by default, some package authors with legitimate needs will stop relying on them. Good ones will document a manual build step. Others will move the work somewhere NPM cannot turn it off — a curl | bash in the install.js, a separate bootstrap binary, a setup command you run after install.*
>
> *The attackers will make the exact same move, because of course they will. If the postinstall hook no longer fires automatically, you don't give up — you find the install path that still executes. This is precisely the kind of threat actor behavior I talked about at DEF CON. Push*

on one control and the malicious activity relocates to where the controls aren't. It doesn't vanish.
And here's the part that should worry NPM but won't, because it's good for them. When the malware moves out of the registry and into a shell script on someone's gist, or a binary downloaded from a CDN, NPM gets to report that registry-resident malware is down. They'll claim victory. The problem will look solved on their dashboard. Meanwhile the risk has simply relocated to terrain with less visibility and fewer tools, not more. The problem gets pushed off the one surface the entire security industry actually instruments, and onto surfaces nobody is watching. That's not a win. That's a measurement artifact.

The third observation is that it gets more difficult for defenders, not easier

Between cool down periods and the disabling of install scripts, large scale NPM attacks will become less frequent. But, when you push legitimate functionality off the well-lit path, you don't just move it — you make it look guilty. A package author who genuinely needs to fetch a platform binary at install time, now doing it through some indirect mechanism to survive the new defaults, produces a fingerprint that looks exactly like evasion. Obfuscated loader, out-of-band fetch, install-time network call to a non-registry host. Five years ago that was a strong malware signal. After v12, a chunk of it is just legitimate software adapting to a stricter world.

So while the number and frequency of the big NPM attacks go down, the signal-to-noise ratio for everyone hunting malicious packages gets worse. The benign and the malicious converge on the same suspicious-looking pattern. We end up triaging a flood of weird-but-fine packages to find the weird-and-actually-bad ones, and the bad ones get better cover precisely because so much legitimate behavior now looks the same way. You bury the needed functionality in something that looks sketchy, and you've built the perfect place to hide a needle — a pattern that looks suss except it isn't. Until it is.

What the firewalls and cooldowns are really telling you

Step back and look at what cooldowns and firewalls actually represent. They're good controls, yes. They're also an admission. The reason a third-party product can flag a malicious version six minutes after it's published is that NPM is not doing it themselves. The reason teams pay for an interception layer in front of the registry is that they can't trust the registry to keep malware out.

We are watching detection and response get pushed onto users and onto a handful of vendors, while the registry that profits from being the default keeps under-investing in its own scanning. The incident cadence so far in 2026 makes the gap obvious. v12 is NPM catching up to the problem set I laid out a year ago. That's progress. But a registry that has been this chronically under-resourced on internal security doesn't get to flip three defaults and call the supply-chain problem handled.

So what actually moves the needle? Where does this leave us? We're left with the unglamorous truth that tooling is necessary and nowhere near sufficient. Turn on cooldowns today — it's the single best ratio of effort to protection available, and there's no reason to wait for v12. Prepare your approve-scripts allowlist now, on 11.16.0+, so the v12 upgrade doesn't break your builds and stampede your team into rubber-stamping everything. If you can manage package firewalls, run them. Do all of it.

I thought this was a terrific piece of "from the field" feedback. As we've been seeing for years, malware that slips into the NPM repository has been steadily increasing and there doesn't appear to be anything that can be done.

The one thought I had while reading this person's posting was that perhaps these changes are being made on the cusp of AI appearing at a time of need. He noted his annoyance that responsibility was effectively being pushed away from the repository and onto less centrally managed external resources and solutions. We see that the largest entities – Cisco and Microsoft jump to mind – have been unable to even police their own internal closed source offerings to rid them of bugs. So the scope of the task for an open source free-for-all repository should not be underestimated. But just as AI is now promising to revolutionize the quality of closed source offerings, it also seems like the perfect solution for repositories such as NPM.

Listener Feedback

Robert Voegtlen

Dear Mr. Gibson, I started listening to Security Now! when I was in fourth grade. I'm now 22 years old and have a degree in Cybersecurity from Augusta University. Security Now! has always been there for as long as I can remember. As I've begun my job search, I've realized just how vast cybersecurity really is. There are so many specialties the more I learn, the more I realize I know nothing. What I've been struggling with is figuring out where to focus. It feels like it could take decades of working across different roles and industries before I discover the niche that truly fits me. By then, much of my career will already be behind me. How do you determine which area of cybersecurity is worth dedicating your life's work to?

Thank you and Leo for everything you've done through Security Now! over the years.
Sincerely, Robert Voegtlen, SpinRite owner.

First, Robert... wow. My question to you would be what could have possibly motivated you as a 4th grader to tune into this podcast? That's really something. And Leo and I certainly have been here throughout nearly all of your life. For that I know that we're both glad and feel very fortunate.

To your question: I have no idea. In my case I sort of stumbled into the security sphere through ShieldsUP!, which I created because there was clearly a need at the time. And then the OptOut! Ad-ware remover, which happened when I discovered that Aureate ad-ware on my own PC. Since my background since about age five, which began with understanding electricity, then expanded into electronics, physics, engineering and computer hardware & software, I had the background to head in pretty much any direction — and at a time when SpinRite was purring along, I saw a need over in the security space.

So I think my best advice would be to perhaps more deliberately do what I had inadvertently done, which is to obtain the widest possible background that you can. That will automatically expose you to many more aspects of the field, and you may find that you naturally gravitate toward something specific. And even if that doesn't happen I believe you'll be better equipped to succeed with whatever you decide to tackle. In today's highly competitive world, I normally advise people to become the best they possibly can at a narrowly targeted specialty. I believe

that's the way to succeed. But that process of specialization can only begin once you're determined what truly interests you.

Todd Whittaker

A college professor listener of ours uses PHP to teach security

Hi Steve, Your recent discussion of insecure PHP in episode 1082 rang very true to me. But you know what? PHP is really quite useful in cybersecurity—just maybe not in the way its defenders usually mean.

I'm a college professor, and when I built our undergraduate cybersecurity major back in 2010, two of the original courses used PHP: Web Programming and Application Security. Clearly, that was not because PHP represented our ideal of secure software design. It was because PHP is almost perfectly suited to showing students how insecure software gets built.

SQL injection? Concatenate user input into raw SQL. Cross-site scripting? Echo unescaped input back to the browser. Broken authentication? Store passwords badly or trust the wrong session variable. PHP makes the mistake easy to write, easy to see, and therefore easy to teach. Once students can see the failure clearly, we can show the corresponding discipline: prepared statements, output encoding, password hashing, access control, session hygiene, and least privilege.

That, for me, is PHP's real value in a security classroom. It gives students a compact, legible catalog of the mistakes they need to recognize before they encounter them in the wild. Many cybersecurity graduates will eventually audit, inherit, or respond to PHP-heavy environments, including Drupal and WordPress installations, where they will need to see past code that merely works and recognize the familiar patterns that make it vulnerable.

And if they leave the courses better equipped to be skeptical of PHP-based platforms in environments where security matters, so much the better. /Todd

I think Professor Todd's use of PHP for teaching about coding security is brilliant. It's an application for PHP that had never occurred to me. I love it. Thanks, Todd!

Derak Kilgo

Hi Steve, 20-year PHP veteran here. I thought you'd find this interesting: The foundation that governs PHP has added a grant funded position to improve PHP's security posture given the realities of AI powered development.
"The PHP Foundation grant will fund a six-month full-time position titled "Ecosystem AI Security Engineer in Residence at the PHP Foundation" to lead this effort and to prepare a sustainability plan for the time after this initial phase. This person will act as a trusted intermediary between security researchers and maintainers in urgent, high-risk situations, and will collaborate with peers in similar roles across other language ecosystems. Additionally, grant funding will also be employed toward the team goals described above where they cannot be accomplished by the single paid lead position or with the help of PHP community volunteers." https://thephp.foundation/blog/2026/05/18/announcing-ecosystem-security-team

I've been listening since you were an actual podcast... on an ipod with a spinning disk.

Love the show. Your perspective is appreciated. / Derak

I think that's a great move overall. But I should be clear, and Todd Whittaker's use of PHP for security education helps to further clarify this: I do not believe that there's anything whatsoever wrong with PHP. It's not at all the language itself I do not trust. There's nothing wrong with the PHP language. I think it's very likely bulletproof. My problem with it – which has been informed through our two decades of covering its use – is that those who often gravitate to PHP do so because it appears to be so easy to use. But we've learned that it's always easier to write code that works than code that works securely. This next bit of listener feedback sets up a perfect example:

The subject of **Steve Meyers'** email was: *"Your PHP rant"*. Steve wrote:

It seemed like your rant against PHP was a bit of a stretch. PHP has its issues, but it's also come a long way. Here are my issues with your rant: You were using some obscure WordPress plugin with a whopping 4,000 installs as the basis for complaining about PHP. Your specific complaint about PHP making it too easy to do bad things is that it has an eval() function.

Here is a list of some other programming languages with eval() functions: JavaScript, Python, Ruby, Perl, Lua, Lisp (specifically noted in Wikipedia as the originator of the eval() function), Scheme, Clojure, and MATLAB.

Anyway, it really seemed like your rant was pretty gratuitous and didn't have a lot to back it up. https://en.wikipedia.org/wiki/Eval

Okay, first of all, Steve is of course correct that the particular problem with that PHP Wordpress add-on was due to its programmer perhaps not being aware of the danger of forwarding user-provided text to an eval() function. And it's certainly true that similar eval() functions exist in many languages. But I'm not upset with PHP for having an eval() function. My concern is that writing secure code for the web is extremely challenging. There are so many different and very subtle ways to screw that up. Professor Whittaker's note mentioned a handful of them and he didn't even mention eval(). So perhaps the best "non-ranty" way of expressing what I mean here is to say that it feels as if there's a larger inherent mismatch between the coding skill of the typical PHP coder, who may be coding for the web for the first time, and the coding skill required to do so securely. It's certainly the case that no sane person will have decided to code their website in LISP. But that said, I'm probably not one to speak since I did choose to code mine in assembly language! :)

Patch Tuesday à la AI

We all knew this had to be coming... and it did not take long... because the new race is now on, to see whether our industry's badly broken software can and will be repaired – with the help of AI – before the bad guys are able to leverage that same AI to find and exploit any of that same software that has not yet been repaired. In other words, what's been expected and predicted with the advancing evolution of AI models focused upon code is all really happening.

Last Tuesday, Microsoft broke their all-time record for the number of security vulnerabilities patched in a single update cycle – and that doesn't count their fixes to their Chromium-based Edge browser which also broke its own record – as our current U.S. president would say: "By a lot!" Those fixes to Edge are now wisely being separated into a separate count. I say "wisely" because if the two counts were not separated, ***June's haul alone*** would land somewhere in the neighborhood of 566 security vulnerabilities repaired!

So let's first take the non-Edge Windows and Windows-adjacent patches. Even without Edge, there are so many that the exact number differs from one report to the next. Counting that high can be tricky. But everyone agrees that it was at least 200 and perhaps as many as 206. Now, we've all become patch-drunk. There was a time when 30 or 40 vulnerabilities would have raised eyebrows. Now that's seen as a quiet month. But stand back for a moment to consider something North of 200 security bugs found and fixed. 200! On the one hand, it's great that Windows and its surrounding software will have at least 200 fewer discoverable security vulnerabilities. On the other hand, Windows software had 200 or more security vulnerabilities to fix; and no one imagines that next month will be much better.

And it's not as if these are insignificant problems that could have been ignored. Oh, no. Among those 200 were six 0-days – five of which had previously been publicly disclosed and one which was under active exploitation in attacks. And among these 200 now-blessedly-resolved Microsoft Windows or Windows-adjacent bugs, 33 of those ranked as "Critical", with all but five of those – so 28 – being remote code execution, 4 of the remaining 5 being privilege elevation and the last critical flaw being an information disclosure. But 28 RCEs found and fixed.

It's nice to see Microsoft moving quickly to employ their Codename MDASH AI to the task of finding and fixing the many flaws we have come to understand Microsoft's code contains. They're moving quickly because they are fully aware that increasingly-capable AI models are also in the hands of malicious actors who are beginning to actively employ those models to the discovery and exploitation of Microsoft's many code shortcomings. So, as I noted at the top of this, it could not be more true that a race is on – it really is.

I still hold that bugs are not inherently endless. Once removed, that bug remains gone. And if similar AI is employed to pre-screen new code before it's released, the past's continuing supply of freshly created ***new*** bugs should finally also be cut off. This means that the consequences of this newly AI-enabled race, which is motivating both sides more than anything ever has, is that we are all going to be receiving far better software from Microsoft than we ever have.

So what's the overall breakdown of these 200-some vulnerabilities? Last Tuesday's more-

worthwhile-than-ever round of updates fixed:

- 65 Elevation of Privilege Vulnerabilities
- 55 Remote Code Execution Vulnerabilities
- 30 Information Disclosure Vulnerabilities
- 27 Spoofing Vulnerabilities
- 19 Security Feature Bypass Vulnerabilities, and
- 7 Denial of Service Vulnerabilities

And just for the record, those did not include flaws repaired in Mariner, Azure HorizonDB, Microsoft Copilot, Copilot Chat, M365 Copilot, Microsoft Exchange Online, and Microsoft Graph, which were repaired previously. In other words, things really are quite furious up there in Redmond, Washington. And that's great for everyone.

So what do we know about the various 0-day vulnerabilities that were fixed last Tuesday?

Thanks to the tireless efforts of the renegade hacker "Nightmare Eclipse" another of their 0-days known as "GreenPlasma" was fixed. That was assigned CVE-2026-45586, an elevation of privilege flaw the disgruntled hacker discovered in Windows Collaborative Translation Framework (CTFMON). It had been publicly disclosed and enabled an unprivileged user account to upgrade itself to full SYSTEM privileges.

Microsoft was also able to quickly repair that HTTP/2 Bomb denial-of-service vulnerability, the deliberate headline-grabbing irresponsible disclosure of which annoyed me so much last week.

The Microsoft variant was assigned CVE-2026-49160 and was found in the HTTP.sys module. As Microsoft phrased it: "Uncontrolled resource consumption in HTTP/2 allows an unauthorized attacker to deny service over a network" – which is Microspeek "for anyone's laptop can bring down our web servers." What we already knew from "Calif's" disclosure OS that the HTTP/2 Bomb is a denial-of-service technique that abuses how the HTTP/2 protocol compresses and manages web traffic headers, allowing attackers to send very small amounts of data that force servers to allocate disproportionately large amounts of memory. By combining two techniques, the researchers at Calif discovered that they could dramatically increase server process memory usage, then keep the memory tied up by manipulating HTTP's flow-control settings to prevent the server from freeing resources. This would result in performance collapse and service outage.

Since this clever attack is more an abuse of deliberate HTTP/2 protocol features rather than a bug that can be fixed, Microsoft also added a new "MaxHeadersCount" registry setting to limit the number of headers in a request. If unspecified, the maximum header count defaults to 200. It can be set as low as 50 or as high as 65,535.

Tuesday's updates also resolved two problems with BitLocker:

Nightmare Eclipse's so-called "YellowKey" vulnerability has been addressed as CVE-2026-45585. Recall that it was the hack that involved rebooting a machine while supplying a script on a thumb drive that had the effect of deleting some files and leaving the system in a pre-booted state with its primary drive decryption key loaded and the normally encrypted drive fully

accessible.
As Microsoft phrased it: "A successful attacker could bypass the BitLocker Device Encryption feature on the system storage device. An attacker with physical access to the target could exploit this vulnerability to gain access to encrypted data." So after last Tuesday's updates, this can no longer be accomplished. Microsoft is now careful to not leave the BitLocker-encrypted drive in an unencrypted state.

So what about the second BitLocker repair? Let's hope the way they stumbled on this one was human-derived and not AI, since it sure feels like "the old Microsoft" rather than the new and improved Microsoft we're hoping AI might be enabling.

This second flaw was named "Bitskrieg" by the guy who discovered it. He originally posted the news of this over on "X" under the name "Jonas L". Since the view from a hacker's perspective is always interesting and often entertaining, I'll share what Jonas originally posted near the start of this month, on June 4th. He wrote:

> *It is rare that anything new happens in the world of IT security – it's mostly just an endless cycle of variants of the same vulnerabilities being exploited again and again. Thats why I appreciate when something new happens, and the YellowKey exploit was, for once, an attack I hadn't seen before.*
>
> *I've myself done a bitlocker bypass before [he supplies a link to a link with an embedded January 15th, 2021 date] – but this one was new to me. It expanded the attack surface onto an area I had not looked into before – the recovery environment.*
>
> *The recovery environment is stored on a partition that bitlocker does not encrypt and the TPM is not locked until you use any functionality that's not the startup repair. So, if you somehow get code running without causing the TPM to lock, you have access to the encrypted drive.*
>
> *Microsoft killed YellowKey by removing the autoexecution of a newly introduced component- that could be manipulated into doing file operations by rolling back a transaction stored on a usb drive. I suspect they simply copied the recovery environment from the unreleased Windows Cloud made for thin clients – that also explained why the bare metal recovery EFI image identifies as Windows 365 when downloading what to boot on its ramdrive from the Microsoft server.*
>
> *The YellowKey transaction rollback hack enabled a file deletion – enabling launching a command prompt by holding down control when launching recenv.exe – an elegant attack. So when my friend asked if I wanted to help try and restore the vulnerability, I figured why not give it a try?*
>
> *Microsoft fixed YellowKey by just killing a specific vulnerability, but they didn't resolve the underlying design issue, so I'd be surprised if it wasn't doable. After 24 hours, a new attack was born – I call it "bitskrieg."*

His posting then walks us through his successful hack and attack which demonstrated that there was, indeed, more than one way to skin that particular cat. And I'll note again, as I did before, that vulnerabilities in BitLocker access at this point is an inherent weakness that Microsoft really cannot do much about. Sure, they could be much less sloppy and more carefully consider the

consequences of their design decisions. But if we want a system that has its Bitlockered drive encrypted at rest, which then autonomously boots into the Windows OS environment that's contained within that BitLockered drive – without requiring some information that is NOT stored on the local machine – which is where the user-supplied PIN comes in – then there's really no way around the fact that the machine will be vulnerable to some form of local-access boot-time shenanigans. There's just no way around that fact.

So, last Tuesday's patch update resolved this additional "bitskrieg" attack that Jonas L discovered. And that's good, since he had made it public. But enterprises and security-minded end users should not rely too heavily upon the security of entirely self-decrypting BitLockered systems. The only way to ever be truly safe is to require some information at boot time that is NOT present anywhere else in the system. That means depending upon a hardware dongle or a manually entered PIN. This is the classic tradeoff between security and convenience and there's no way around it.

Okay, so what else do we know about last Tuesday's record breaker?

The history behind this next 0-day is curious because it's a fix for a CVE dated six years ago in 2020. CVE-2020-17103, which is a Windows Cloud Files Mini Filter Driver Elevation of Privilege Vulnerability. Like so many other recently released 0-days, this one, too, owes our attention to it to none other than "Nightmare Eclipse". Its reincarnation by that now-infamous hacker was given the name "Mini-Plasma".

So what's with the CVE from six years ago? Nightmare Eclipse explained that the flaw was originally reported to Microsoft by Google's Project Zero. And, indeed, I found the original bug with its full description and an attached proof of concept in a ZIP file. However, Nightmare Eclipse stated that the flaw was still exploitable, and it was unclear whether Microsoft fully patched the issue or whether the bug may have been reintroduced at some point. In any event, it appears to have been fixed once again.

And this brings up to the 5th and final 0-day remote code execution vulnerability. Just to be clear, this is one of the 55 remote code execution vulnerabilities that were fixed last Tuesday, though the majority, while RCEs, were not 0-days. But this one, CVE-2026-42897, was. This last one is a spoofing vulnerability that was present in Microsoft Exchange Server. It was being actively exploited in the wild to execute JavaScript in a targeted victim's browser. Microsoft explained: "An attacker could exploit this issue by sending a specially crafted email to a user. If the user opens the email in Outlook Web Access and certain interaction conditions are met, arbitrary JavaScript can be executed in the browser context."

Technically, this last one was not part of the primary patch Tuesday bundle. At the time, Microsoft explained that they were still working on its update and that they would be pushing it out through the Exchange Emergency Mitigation Service, which should be enabled by default on Exchange Server – for just such a need. We know nothing about who disclosed this vulnerability nor how it was exploited.

Scanning down the surprisingly lengthy list of CRITICAL remote code vulnerabilities, we find that one was present in Active Directory Domain Services and another in Microsoft Azure Kubernetes

Service. Microsoft Office and the Remote Desktop Client had seven each and there was one in Nuance PowerScribe. Three were found and removed from Windows Hyper-V. There was one in Windows Deployment Services. Windows DHCP Client had one – that's a scary place to have a critical RCE, since most Windows clients will be using DHCP, so their client will be vulnerable remotely by default. In addition to the Windows HTTP/2 Bomb denial of service problem that was fixed, the HTTP.sys module also had a terrifying critical remote code execution vulnerability fixed. I assume it was terrifying, since it was in HTTP.sys, which is the Windows web server, it was rated critical, and nothing is more exposed than a web server – which is why the HTTP/2 Bomb vulnerability was such a problem. Windows Kerberos also had a critical RCE, and not to be left out, the Windows Kernel did, too. And finally, two RCEs were found and fixed in the Windows Win32K Graphic GRFX module.

So, whew! Yeah. I named today's podcast "Patch Tuesday à la AI" because this is what the next several months or so are likely to look like. It's going to be very interesting to see the shape of the vulnerability discovery and remediation curve. As we know, the previous month's Patch Tuesday tied for the most patches in any month. And this month's handily exceeded that one. What will next month's look like?

Finally, over on the Microsoft Edge/Chromium updates side, I'll quote from BleepingComputer's reporting about that, which wrote: *"There were also a massive 360 Microsoft Edge/Chromium flaws that were fixed by Google this month."* **360!!** Three hundred and sixty!! And these were found in the Chromium browser which was already the recipient of an incredible expenditure of past manpower. But now our entire software industry is replacing manpower with AI-power as rapidly as it can, and it's quickly becoming clear that at least in the field of software, there's really no comparison – man versus machine – since the results are pretty much speaking for themselves – machine wins.

I, for one, cannot wait to see what happens next!